

MINECRAFT SNAKE GAME

May Syi Ang - Electrical and Computer Engineering

Kashvi Jayagiri - Computer Engineering and Computer Science

EE354L - Fall 2025

Lab Session - Monday, 6pm - 9pm, Shri Krishnamachari

December 1, 2025

I. Abstract

Our semester project involves implementing the snake game on an FPGA with Minecraft visuals. Our design utilises three main Verilog files: the state machine file, the data path unit file, and the top file. We use a 15x15 grid as our gameplay area. Users win by achieving the snake's maximum length of 225, and lose if the snake's head hits the boundaries of the gameplay area or collides with itself.

II. Introduction and Background

Using our GCD files from the September 29 lab as a template, we build the gameplay logic state machine, data path unit, and top file from there, heavily modifying them while keeping the syntax relatively similar. To implement the VGA display, we referred to the demo files provided for this project and made the necessary modifications. All modules created were instantiated in the top file, and we simulated the top file using a testbench modified from the GCD lab.

To ensure that the button presses are functional and not glitchy, we utilized the provided debounce file from previous labs to generate SCEN pulses whenever a button is pressed.

III: The Design

(A) Design description of your implementation

Our design is based on a simple state machine that we elaborate on in part B of this section. Using that as a guideline, we developed the Verilog code and instantiated our state machine and data path unit modules in the top file.

On the whole, our design uses a circular buffer to keep track of the head, body, and tail parts of the snake. On each clock, a new head is inserted, and the old tail is removed. If an apple is eaten, a new head is inserted but the old tail stays as the tail, effectively increasing the length of the snake. The circular buffer uses an array of 225 registers, each storing up to 8 bits (Cell_Snake). The 8 bits are the concatenated {x, y} coordinates within the 15x15 grid. If for

example, the snake length is only 4 units, then 221 out of the 225 registers will be 8'hFF which tells us that the snake part does not yet exist because it has not grown to that length.

Note: Formula used to build the relationship between cell coordinates and the index of our arrays:

$$\text{index} = x * 15 + y.$$

For instance, (1,5) will correspond to index 20.

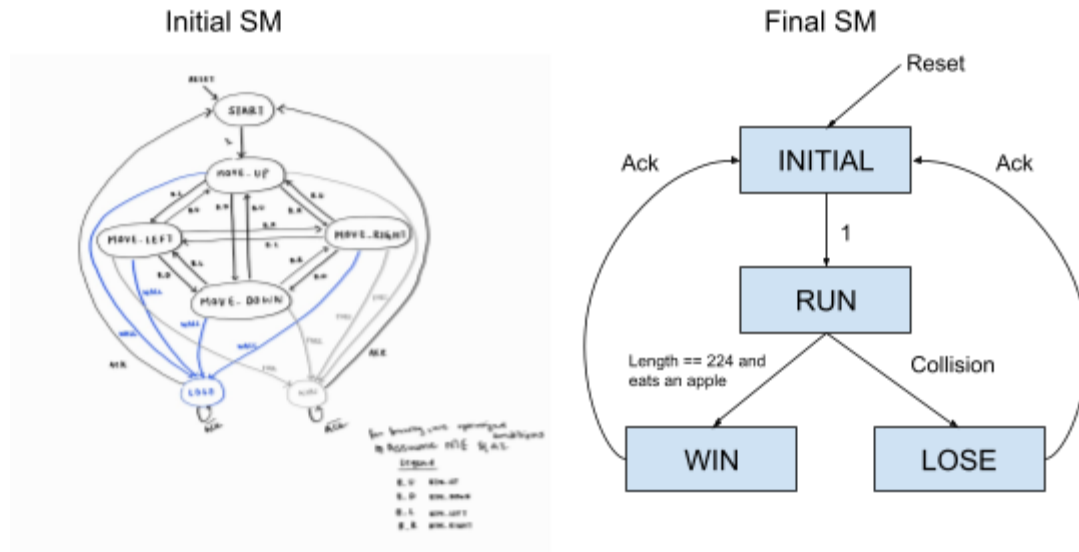
A flattened version of this array is also used, where a single register stores 225 bits of data (Cell_Snake_Vector). Each bit represents a cell in the gameplay area, listed from 0 to 224, from left to right, and from top to bottom. If the bit is a 1, it means that a snake part exists in the cell. Using this flattened array, we can check for the presence of the snake more efficiently in our two main data path modules: the apple generator (ee354_project_apples) and the position calculator (ee354_project_length).

The apple generator uses information from the flattened vector to generate apples in cells that are not occupied by the snake. Upon choosing a cell to generate a new apple in, that binary cell number is then converted into {x, y} coordinates that the VGA module uses to display a sprite of the apple. In order to “randomly” generate the apple positions, we used a method called Linear Feedback Shift Register, which we found online, to generate pseudo-random numbers.

The position calculator is a large module that updates the length of the snake, the position of snake parts as it moves or lengthens, and also checks for collisions between the snake head and its body or the boundaries.

(B) Explanation of the state machine

We began work by designing a state machine. Initially, we had a more complicated state machine, which consisted of states for each direction the snake is moving in. However, we simplified the state machine later when developing the code to just consist of INITIAL, RUN, LOSE, and WIN states. Do note that the following state machine does not account for All Inclusivity for brevity of the picture although it is Mutually Exclusive.



In our gameplay, the game starts immediately upon pressing the Reset button, as we immediately go into the Run state after the Initial. However, due to the simplified design, the data path unit code would be a little more complex. Nevertheless, we decided to stick with the simplified state machine, as making each direction its own state would have resulted in a lot of code repetition. Although, it is important to note that we did use bits to represent directions for each cell to tell us which direction the snake should move.

(C) User Interface

Users can play our game by pressing the up, down, left and right buttons on the FPGA to control the snake's direction of travel. The center button is used as Ack and Reset buttons. Each push button is debounced and produces a SCEN pulse that our top and data path files use to determine snake behavior and the current/next game state. The debouncer module uses the maximum 100 MHz clock (sys_clk) to calculate the duration of button presses and determine whether an SCEN pulse should be generated.

Our actual game implementation using the state machine and data path modules, however, uses a ~5 MHz clock (Speed_Clk = DIV_CLK[24]) to allow the snake to move at a slower and more user-friendly speed. This is also the rate at which our VGA display "changes" as it receives information at this frequency. The display controller module (display_controller) generates VGA timing while block_controller consumes data from the data path and state machine files to produce 12-bit RGB information.

The LEDs on the FPGA tell us which state the game is in. If LED 1 is lit up, the game is in the INITIAL state. LED 2 means the game is in RUN state, LED 3 means it is in LOSE state and LED 4 means it is in WON state.

The SSDs on the FPGA display the number of points the user has scored, which also corresponds to the length of the snake. The points are displayed in decimal, and we scan the SSD using `ssdscan_clk` (`DIV_CLK[19:18]`) which lights up the SSD panels in succession quickly enough to prevent visible flicker to the human eye.

Our VGA displays the movement of the snake and the visuals by passing in relevant information in our top file. We pass into our VGA module the `Cell_Snake_Vector` that helps in determining which cell in our 15x15 grid has a snake component. We also pass in the cell numbers occupied by the head and tail pointers of the snake to ensure we use our assets in the right places.

IV. Test methodology:

For our testbench, I run 7 mini tests to make sure the state transitions are as expected. In test 1, I simply test the reset button by simulating a `BtnC` press. In test 2, I wait for the `INITIAL` to `RUN` state transition that happens necessarily after the `INITIAL` state is reached. In test 3, I simulate a `BtnR` press and ensure that the game remains in `RUN` state. In test 4, I allow the snake to continue moving for the amount of time needed for it to move 8 cells right and hit the boundary. As such, the collision signal should be generated, and the game should reach `LOSE` state.

From test 5 onwards, we reset the game and test whether the game reaches `LOSE` state after colliding into itself. To execute this, we allow the snake to move right, then move left.

In this testbench, we set a 90 ms delay between the press and release of a button to enable the generation of an `SCEN` signal. Additionally, we consider ~340 ms to be the period of `Speed_Clk`, which is the amount of time needed for the snake to move one cell, or to register a collision. The results are as shown below.

```

# EE354L Project Testbench Starting...
# -----
# Test 1: Reset and Initial State
#   Applying reset...
#   TEST:      Ld3=0, Ld2=0, Ld1=0, Ld0=1
#   EXPECTED: Ld3=0, Ld2=0, Ld1=0, Ld0=1 (Initial state)
#
# Test 2: Initial to run state
#   Waiting for state transition...
#   TEST:      Ld3=0, Ld2=0, Ld1=1, Ld0=0
#   EXPECTED: Ld3=0, Ld2=0, Ld1=1, Ld0=0 (Run state)
#
# Test 3: Move right
#   Pressing right button...
#   Button released
#   TEST:      Ld3=0, Ld2=0, Ld1=1, Ld0=0
#   EXPECTED: Ld3=0, Ld2=0, Ld1=1, Ld0=0 (Lose state)
#
# Test 4: Long Running Simulation
#   Running simulation...
#   Simulation complete
#   TEST:      Ld3=0, Ld2=1, Ld1=0, Ld0=0
#   EXPECTED: Ld3=0, Ld2=1, Ld1=0, Ld0=0 (Lose state)
#
# Test 5: Lose and Initial State
#   Applying reset...
#   TEST:      Ld3=0, Ld2=0, Ld1=0, Ld0=1
#   EXPECTED: Ld3=0, Ld2=0, Ld1=0, Ld0=1 (Initial state)
#
..

```

```

# Test 6: Initial to run state
#   Waiting for state transition...
#   TEST:      Ld3=0, Ld2=0, Ld1=1, Ld0=0
#   EXPECTED: Ld3=0, Ld2=0, Ld1=1, Ld0=0 (Run state)
#
# Test 7: Move right and left to lose
#   Pressing right button...
#   Button released
#   Pressing left button...
#   Button released
#   TEST:      Ld3=0, Ld2=1, Ld1=0, Ld0=0
#   EXPECTED: Ld3=0, Ld2=1, Ld1=0, Ld0=0 (Lose state)
# -----
# Testbench Complete

```

V. Conclusion and Future Work:

Although we've implemented most of the original features in our game, there are some enhancements that students in future semesters can implement and that we hope to implement ourselves one day too. Our current state machine for this game only lets the user win if the snake fills up the grid which is almost impossible to attain because of body and wall collisions. Instead, there can be a more plausible winning condition, for instance, reaching length 100. They can also implement a start, winning and losing screen UI visuals. One other thing that future work can implement is increasing the speed of the snake as it grows in length and eats more apples to make it more challenging to win. Furthermore, this game can be enhanced by trying to implement a design using fewer than 225 registers.

Looking back, the labs this semester were very fulfilling to carry out. They were challenging enough for us to think for ourselves. We were still given the knowledge and information to carry these labs out, though, through lectures and material given in the lab decks. The TAs were very resourceful and helpful with labs, especially Shri. Some labs were also more time-consuming and difficult than the others. The lab where we updated code for a 4bit divider into an 8-bit divider was particularly difficult to do because there were many aspects that the lab dealt with. We really enjoyed the VGA lab as it was wonderful to see how you can visually represent state machines using code and hardware. One facet that would probably make the labs better was the organization of material. Some time from our labs would go into trying to find where the material for the week was. Categorizing files and naming the labs appropriately (according to week or lab number) would definitely have made

navigating the lab easier. Overall though, the labs were well structured, well supported and set us up for success in our final project.